

Matteo Palmonari

matteo.palmonari@unimib.it

*Representation Learning,
Distributional Semantics,
Language Models*



dat·AI

data * AI
AI * data

3 Key Concepts Towards Transformers/LLMs

- Representation learning
 - Representations learned from data
 - distributed representations/embeddings
- Distributional semantics
 - A different approach to semantics
 - meaning is related to language distributions
- Language modeling
 - Probabilistic model of language
 - a model that predicts the next word after a word sequence



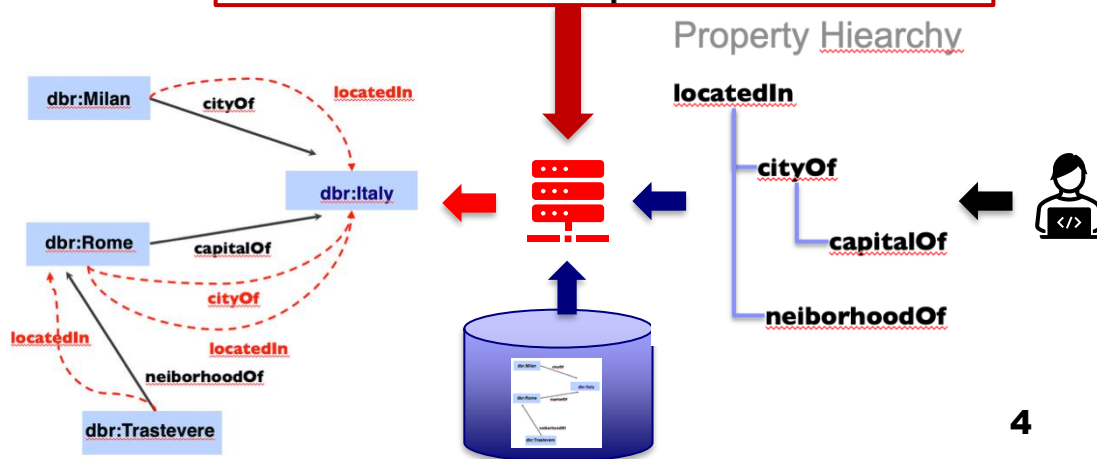
Introduction to representation learning, recap of Deep Neural Networks (DNNs)

REPRESENTATION LEARNING AND DNNs

Rule-based AI

- The **knowledge engineer**  specifies the **axioms** that capture a **domain theory** (e.g., TBox)
- **Data** are encoded as ground facts (e.g., ABox)
- A general **reasoner**  computes the inferences according to a **truth theory**
 - **Inferences**: what must be true assuming a set of premises
- Every inference is **explainable** (proof)

if	then
<code>x rdfs:subClassOf y .</code> <code>a rdfs:type x .</code>	<code>a rdfs:type y .</code>
<code>x rdfs:subClassOf y .</code> <code>y rdfs:subClassOf z .</code>	<code>x rdfs:subClassOf z .</code>
<code>x p y .</code> <code>p rdfs:subPropertyOf q .</code>	<code>x q y .</code>
<code>p rdfs:subPropertyOf q .</code> <code>q rdfs:subPropertyOf r .</code>	<code>p rdfs:subPropertyOf r .</code>
<code>x p y .</code> <code>p rdfs:domain z .</code>	<code>x rdfs:type z .</code>
<code>x p u .</code> <code>p rdfs:range z .</code>	<code>u rdfs:type z .</code>



Representation Learning - KGs

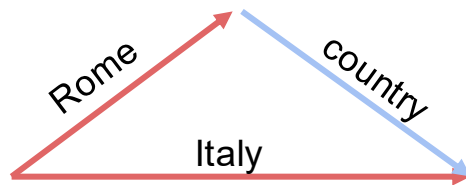
- We let the machine – usually some DNN - learn the representations of symbols that optimize some task
- Representations are vectors – usually in $\mathbf{R}^n$ – whose individual values are not individually interpretable
- Learned representations optimize the learned task but often show also other remarkable regularities

- Example with Knowledge Graphs (TransE):

- A training set of facts:
 - $\langle \text{subject}, \text{predicate}, \text{object} \rangle$
- Optimization task:
 - $\mathbf{v}(\text{subject}) + \mathbf{v}(\text{predicate}) \approx \mathbf{v}(\text{object})$
- What we get:

$$\mathbf{v}(\text{Rome}) + \mathbf{v}(\text{country}) \approx \mathbf{v}(\text{Italy})$$


4	3	6	2	3
+				
1	2	1	5	2
≈				
2	3	5	1	7

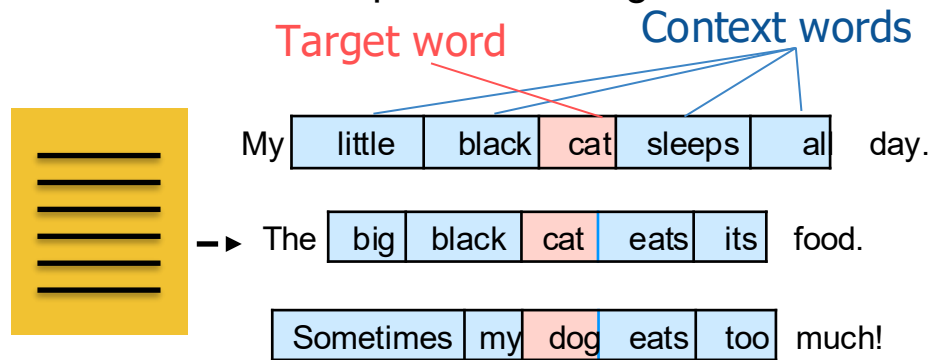


Representation Learning - Words

- We let the machine – usually some DNN - learn the representations of symbols that optimize some task
- Representations are vectors – usually in $\mathbf{R}^n$ – whose individual values are not individually interpretable
- Learned representations optimize the learned task but often show also other remarkable regularities

- Example with word representations (Word2vec):

- A text corpus as training set:



- Optimization task:
 - Predict the target word given the context words
- What we get:



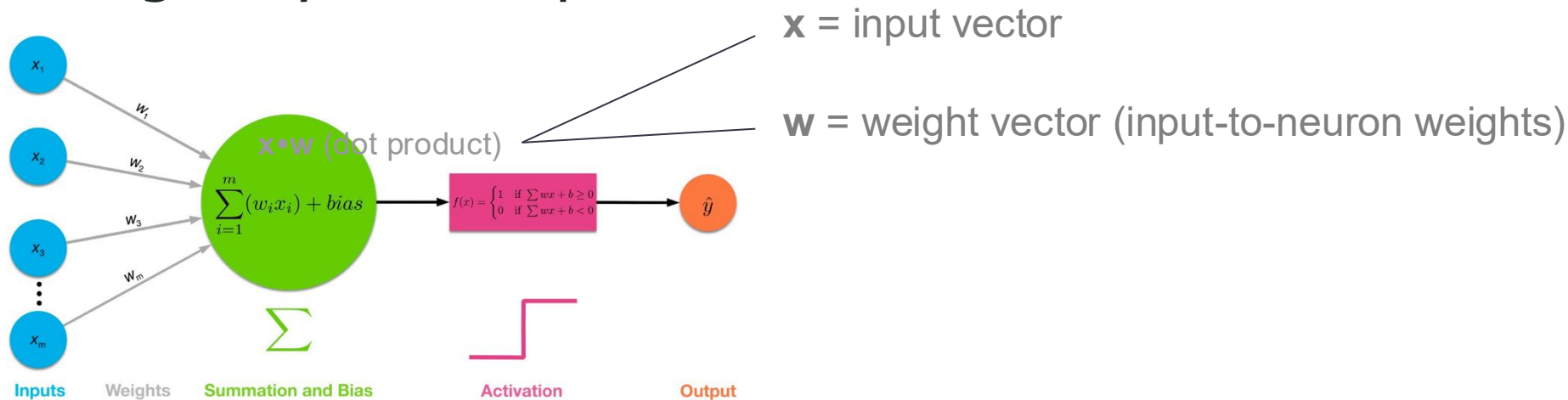
Representation Learning x DNNs

Why DNNs support representation learning from the data?
A quick recap of DNNs

Neural Networks (NNs)

- Inspired by neurons in our brains but very different
- Function approximators
 - Process input signals, return output signals
 - $\text{NN}(\text{input_signal}) = \text{output_signal}$
 - Learn to approximate functions from data
 - Data: usually input-output maps

Single-layer Perceptron

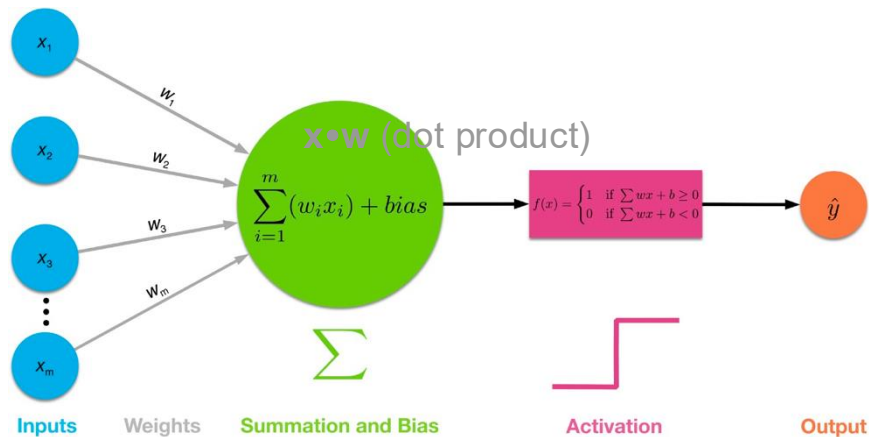


- We often use $\mathbf{x}$ to refer to the vector $\langle x_1, \dots, x_m \rangle$
- Bias can be omitted (i.e., $bias = 0$)
- Several activation functions can be used ... or no activation function

Images from this nice intro to (Deep) Neural Networks (recommended):

<https://towardsdatascience.com/multi-layer-neural-networks-with-sigmoid-function-deep-learning-for-rookies-2-bf464f09eb7f>

Single-layer Perceptron



- We often use $\mathbf{x}$ to refer to the vector $\langle x_1, \dots, x_m \rangle$
- Bias can be omitted (i.e., $bias = 0$)
- Several activation functions can be used ... or no activation function

Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = \max(0, z)$	Multi-layer Neural Networks	
Rectifier, softplus	$\phi(z) = \ln(1 + e^z)$	Multi-layer Neural Networks	

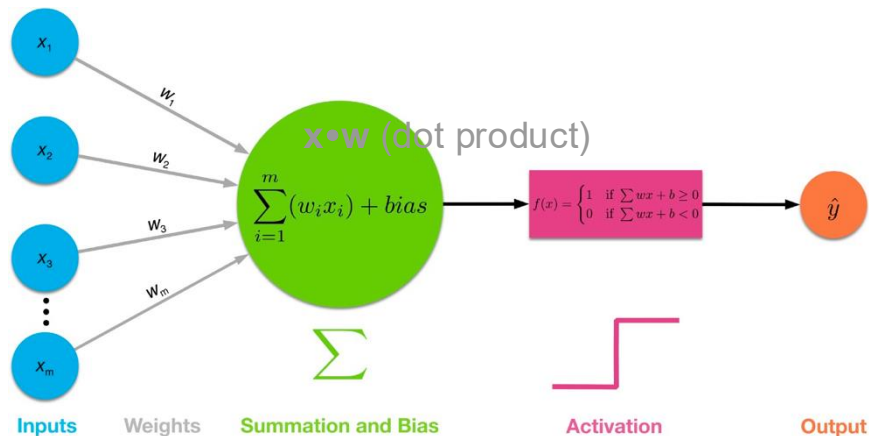
Copyright © Sebastian Raschka 2016
(<http://sebastianraschka.com>)

List of activation functions by [Sebastian Raschka](#)

Images from this nice intro to (Deep) Neural Networks (recommended):

<https://towardsdatascience.com/multi-layer-neural-networks-with-sigmoid-function-deep-learning-for-rookies-2-bf464f09eb7f>

Single-layer Perceptron



- We often use $\mathbf{x}$ to refer to the vector $\langle x_1, \dots, x_m \rangle$
- Bias can be omitted (i.e., $bias = 0$)
- Several activation functions can be used ... or no activation function

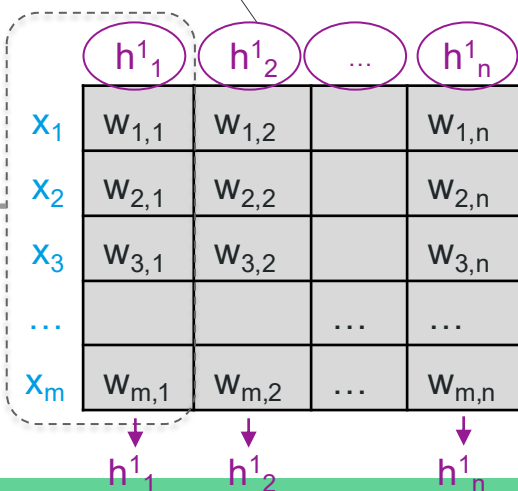
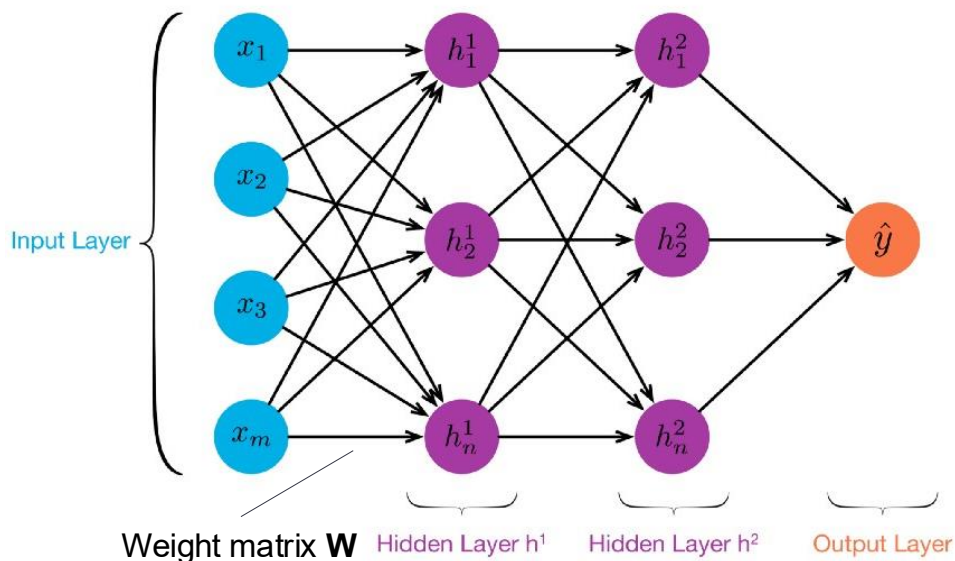
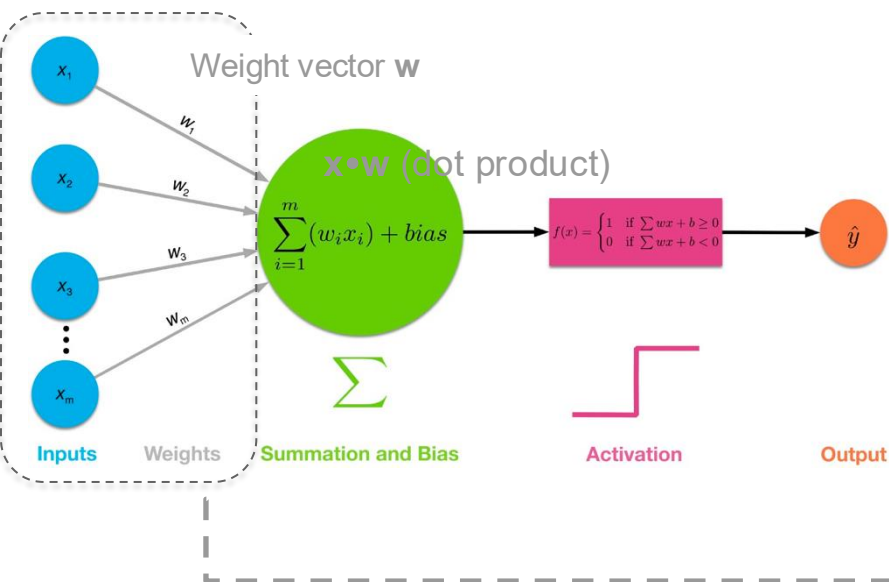
Learn to approximate functions from data:

- Weights are **initialized**
- Weights are **updated** until convergence or stop
 - **Forward pass:** input from training data are passed through the network and the output is computed
 - **Error estimation:** error of predicted vs desired output is computed
 - **Backward pass:** weights are updated to minimize error
 - Most famous approach: backpropagation

Images from this nice intro to (Deep) Neural Networks (recommended):

<https://towardsdatascience.com/multi-layer-neural-networks-with-sigmoid-function-deep-learning-for-rookies-2-bf464f09eb7f>

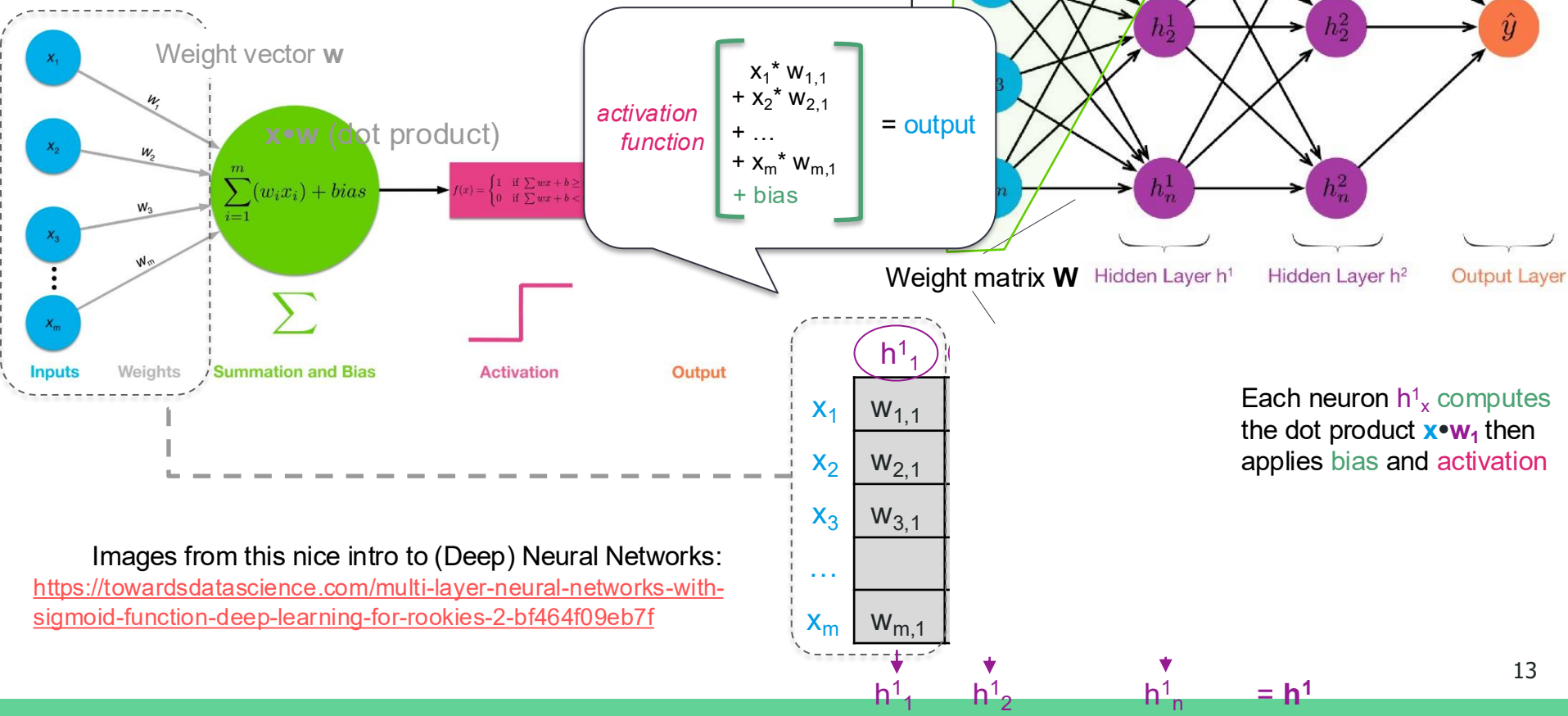
Multi-layer Perceptron



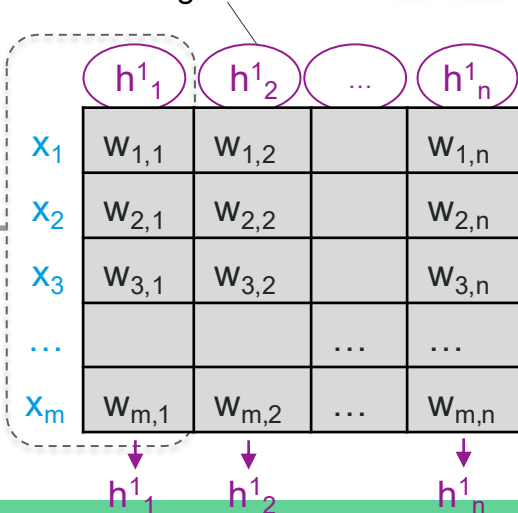
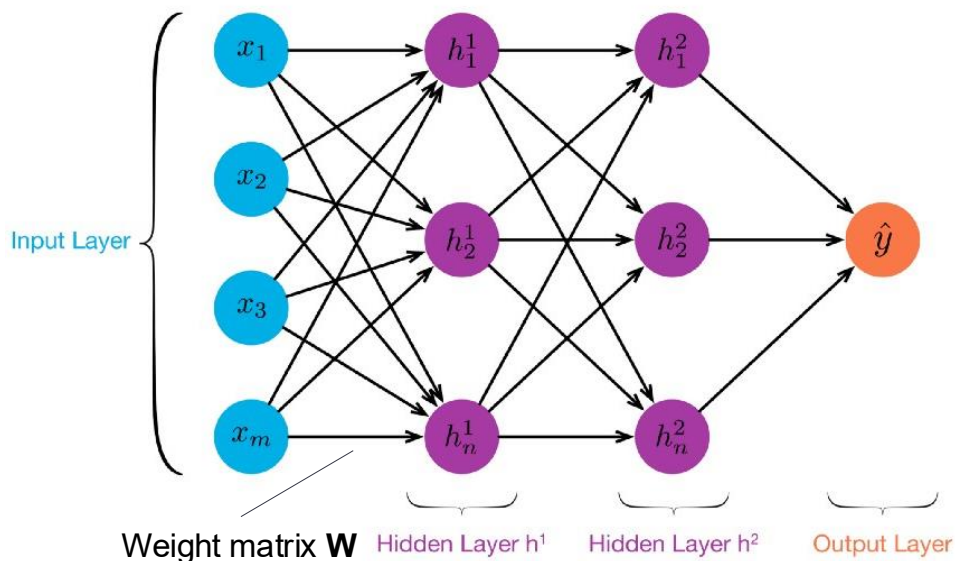
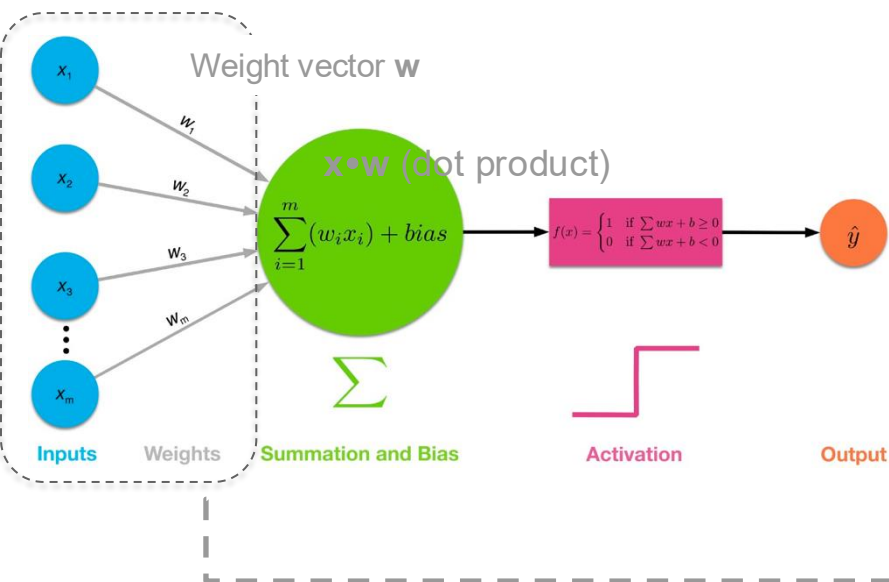
Each neuron h^1_x computes the dot product $x \cdot w_1$ then applies bias and activation

Images from this nice intro to (Deep) Neural Networks:
<https://towardsdatascience.com/multi-layer-neural-networks-with-sigmoid-function-deep-learning-for-rookies-2-bf464f09eb7f>

Multi-layer Perceptron



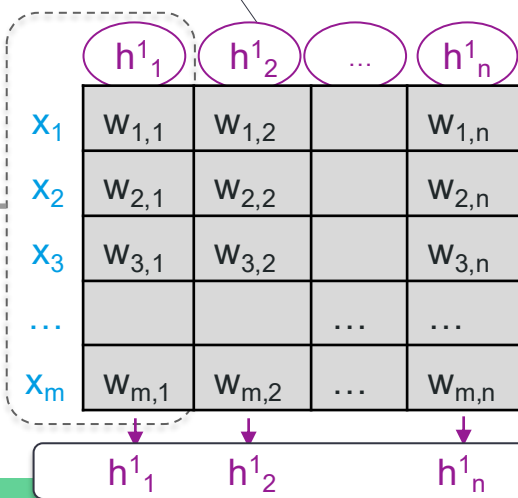
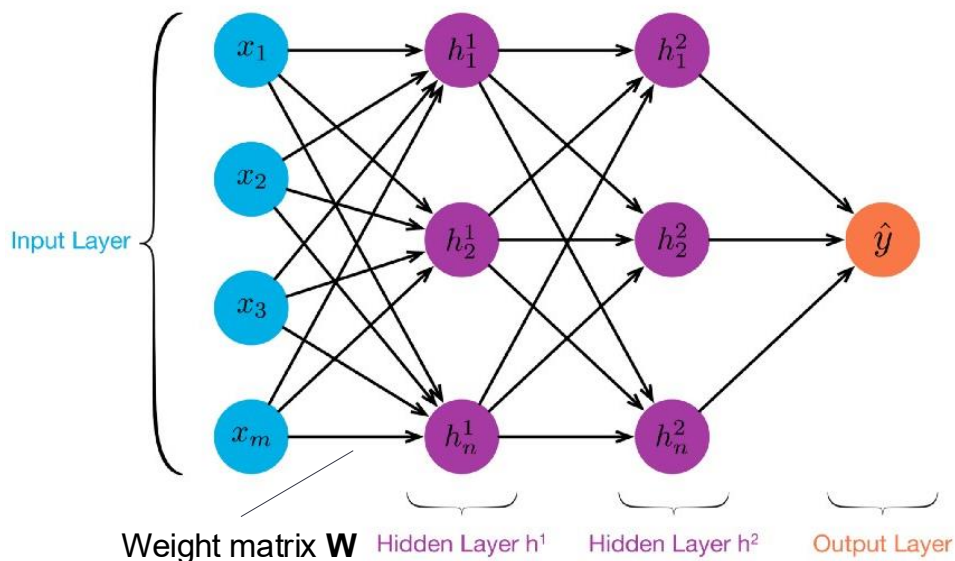
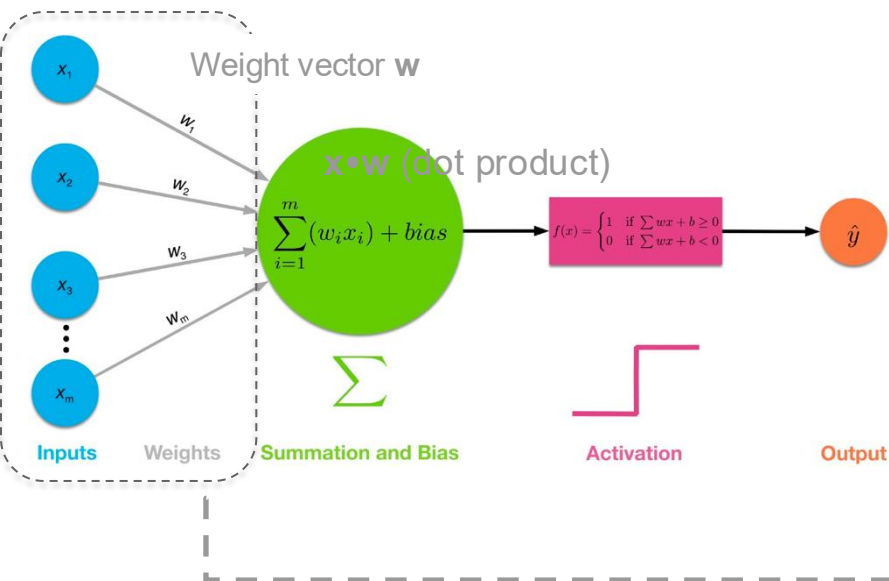
Multi-layer Perceptron



Each neuron h_x^1 computes the dot product $x \cdot w_x$ then applies bias and activation

Images from this nice intro to (Deep) Neural Networks:
<https://towardsdatascience.com/multi-layer-neural-networks-with-sigmoid-function-deep-learning-for-rookies-2-bf464f09eb7f>

Multi-layer Perceptron

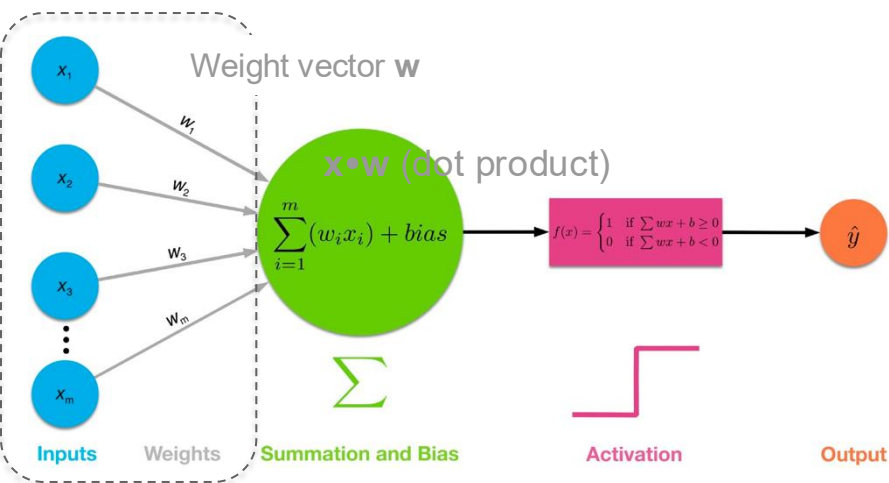


Each neuron h^1_x computes the dot product $\mathbf{x} \cdot \mathbf{w}_1$ then applies bias and activation

Each layer h^y applies these operations to the output vector of the previous layer $\mathbf{x}^{y-1}$ and using its weight matrix $\mathbf{W}^y$

Images from this nice intro to (Deep) Neural Networks:
<https://towardsdatascience.com/multi-layer-neural-networks-with-sigmoid-function-deep-learning-for-rookies-2-bf464f09eb7f>

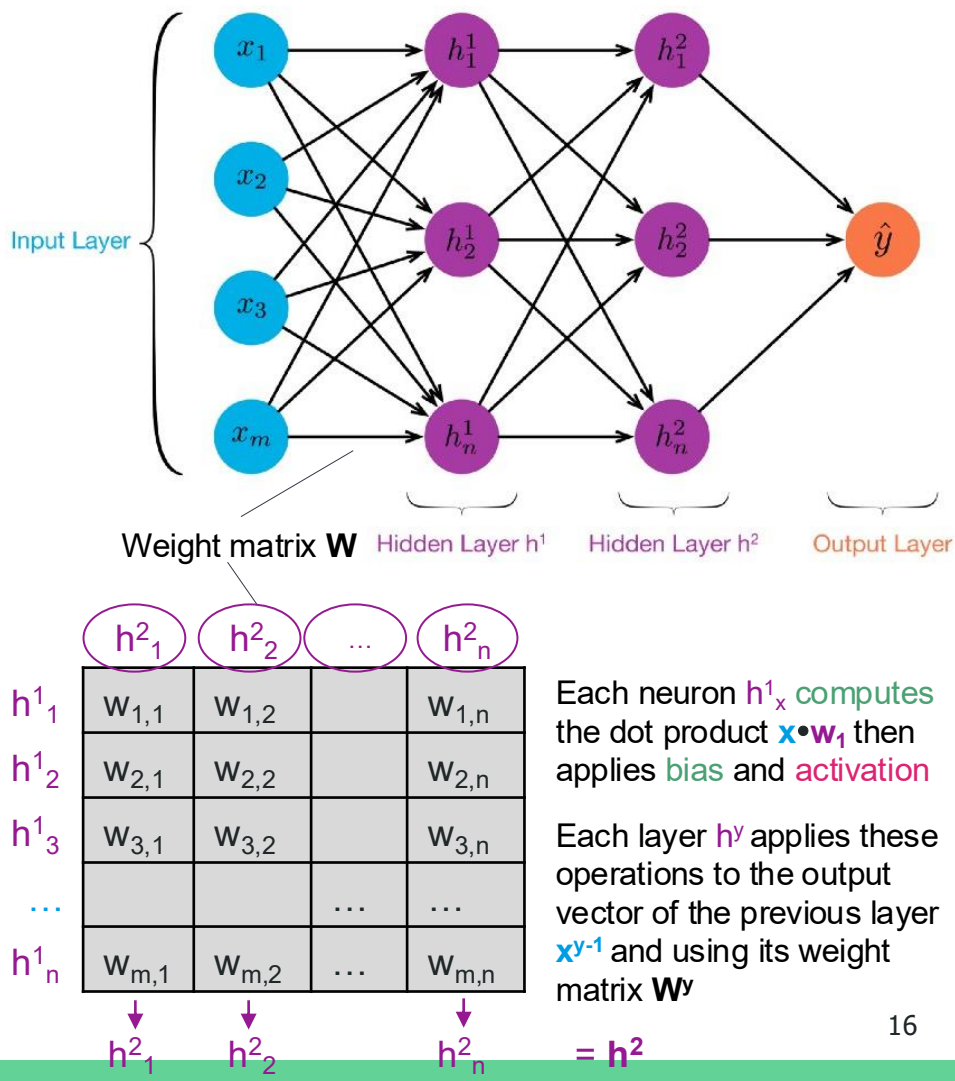
Multi-layer Perceptron



In practice,
the network multiplies matrices throughout the layers
(with bias and activation functions)

Images from this nice intro to (Deep) Neural Networks:

<https://towardsdatascience.com/multi-layer-neural-networks-with-sigmoid-function-deep-learning-for-rookies-2-bf464f09eb7f>



DNNs Recap

- All NNs: function approximators
 - Process input signals, return output signals
 - Learn to approximate functions from data
- Shallow NNs
 - Features usually engineered and included in the input
 - The network learns how to weight input features
- Deep NNs
 - The network learn to extract features: representation learning
 - Multiple weight matrices across (hidden) layers
 - With many layers, often hierarchical features: more semantic features towards the output
 - More weights requires update, more data are needed

Distributions vs. definitions / rules

DISTRIBUTIONAL SEMANTICS

Distributional Semantics (Computational Linguistics)

Distributional Semantic Models

Part 1: Introduction

Stefan Evert¹

with Alessandro Lenci², Marco Baroni³ and Gabriella Lapesa⁴

¹Friedrich-Alexander-Universität Erlangen-Nürnberg, Germany

²University of Pisa, Italy

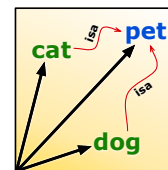
³University of Trento, Italy

⁴University of Stuttgart, Germany

<http://wordspace.collocations.de/doku.php/course:start>

Copyright © 2009–2016 Evert, Lenci, Baroni & Lapesa | Licensed under CC-by-sa version 3.0

Check slides & video lessons
Part 1 & 2



Distributional Semantics in a Nutshell #1

What's the meaning of '**bardiwac**'? (Lenci & Evert)

- He handed her a glass of **bardiwac**.
- Beef dishes are made to complement the **bardiwacs**.
- Nigel staggered to his feet, face flushed from too much **bardiwac**.
- Malbec, one of the lesser-known **bardiwac** grapes, responds well to Australia's sunshine.
- I dined on bread and cheese and this excellent **bardiwac**.
- The drinks were delicious: blood-red **bardiwac** as well as light, sweet Rhenish.

Distributional Semantics in a Nutshell #1

What's the meaning of '**bardiwac**'? (Lenci & Evert)

- He handed her a glass of **bardiwac**.
- Beef dishes are made to complement the **bardiwacs**.
- Nigel staggered to his feet, face flushed from too much **bardiwac**.
- Malbec, one of the lesser-known **bardiwac** grapes, responds well to Australia's sunshine.
- I dined on bread and cheese and this excellent **bardiwac**.
- The drinks were delicious: blood-red **bardiwac** as well as light, sweet Rhenish.

→ '**Bardiwac**' is a heavy red alcoholic beverage made from grape

Accounting for the Meaning of Language Expressions #2

■ Distributional Semantics

- “The meaning of a word is its **use in the language**”
— Ludwig Wittgenstein, 1953
- “You shall know a word by **the company it keeps!**”
— J. R. Firth, 1957
- Distributional hypothesis: difference of meaning correlates with difference of distribution
— Zellig Harris, 1954
- “What people know when they say that they know a word **is not how to recite its dictionary definition** – they **know how to use it** [. . .] in everyday discourse.”
— Miller, 1986

Word Collocations

What is the meaning of “bardiwac”?

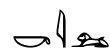
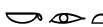
bardiwac British National Corpus freq = 230

object of 32 1.5	and/or 47 1.7	pp_obj round-p 1 29.1	pp_obj of-p 63 5.7	pp_obj through-p 1 4.5
uncork 1 8.98	plummy 1 9.33	pass 1 0.3	swig 1 7.21	plausible 1 5.28
gulp 1 6.61	Sancerre 1 9.14		tinge 1 6.44	
sport 1 5.6	Willson 1 8.93	pp before-p 1 13.0	bottle 24 6.35	predicate of 4 3.7
water 1 5.34	scampi 1 8.23	dinner 1 1.98	goblet 1 6.29	Branaire-ducru 1 12.19
drink 7 5.13	burgundy 1 8.18		jug 1 4.64	Spar 1 8.85
sip 1 4.8	garb 1 7.02	pp_obj after-p 1 6.5	grape 1 4.63	liquor 2 5.82
warm 1 4.28	ruby 1 6.59	sought 1 8.56	cup 16 4.38	
complement 1 4.15	Barnett 1 5.29		bowl 2 3.66	
waste 1 2.93	refreshment 1 5.29		glass 4 2.83	
paint 1 2.38	Halifax 1 5.11		label 1 2.76	

pp_obj with-p 6 3.3	pp_obj by-p 4 2.5	predicate 2 1.8	pp_obj from-p 2 1.6	modifier 72 1.2
fagg 1 9.54	embolden 1 8.29	tipple 1 7.91	burgundy 1 8.91	passable 5 9.92
brim 1 6.71	refresh 1 6.36	wine 1 1.53	flush 1 4.71	ready-to-drink 1 8.79
stain 2 5.49	confuse 1 4.36			cinnamon-scented 1 8.79
merchant 1 2.68	accompany 1 1.63	pp_obj to-p 5 1.7	adj subject of 3 1.2	rust-coloured 1 8.57
meal 1 1.64		alternative 1 2.2	cheap 1 3.08	Tanners 1 8.51
	pp as-p 1 1.9	trip 1 1.7	happy 1 1.66	ten-man 1 8.43
	gift 1 2.14	attend 1 1.35	sure 1 0.56	in-flight 1 7.99
				full-bodied 1 7.87
				Smedley 1 7.83
				blood-red 1 7.75

Word Collocations

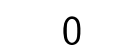
A thought experiment: deciphering hieroglyphs

							
(knife)		51	20	84	0	3	0
(cat)		52	58	4	4	6	26
???		115	83	10	42	33	17
(boat)		59	39	23	4	0	0
(cup)		98	14	6	2	1	0
(pig)		12	17	3	2	9	27
(banana)		11	2	2	0	18	0

verb-object counts from British National Corpus

Word Collocations

A thought experiment: deciphering hieroglyphs

								
(knife)		51	20	84	0	3	0	$\text{sim}(\text{knife icon}, \text{boat icon}) = 0.770$
(cat)		52	58	4	4	6	26	$\text{sim}(\text{knife icon}, \text{cat icon}) = 0.961$
???		115	83	10	42	33	17	
(boat)		59	39	23	4	0	0	
(cup)		98	14	6	2	1	0	
(pig)		12	17	3	2	9	27	$\text{sim}(\text{knife icon}, \text{banana icon}) = 0.939$
(banana)		11	2	2	0	18	0	

verb-object counts from British National Corpus

Word Collocations

English as seen by the computer ...

		get 	see 	use 	hear 	eat 	kill 
knife		51	20	84	0	3	0
cat		52	58	4	4	6	26
dog		115	83	10	42	33	17
boat		59	39	23	4	0	0
cup		98	14	6	2	1	0
pig		12	17	3	2	9	27
banana		11	2	2	0	18	0

$$\text{sim}(\text{knife}, \text{dog}) = 0.770$$

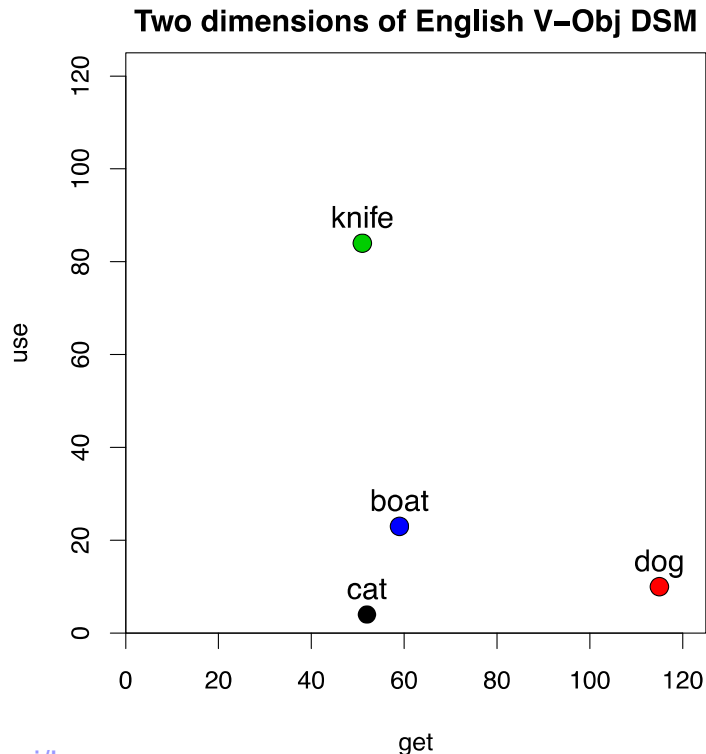
$$\text{sim}(\text{cat}, \text{dog}) = 0.961$$

$$\text{sim}(\text{dog}, \text{banana}) = 0.939$$

verb-object counts from British National Corpus

Distributed Representations: Geometric Interpretations #1

- ▶ row vector $\mathbf{x}_{\text{dog}}$ describes usage of word *dog* in the corpus
- ▶ can be seen as coordinates of point in n -dimensional Euclidean space
- ▶ illustrated for two dimensions: *get* and *use*
- ▶ $\mathbf{x}_{\text{dog}} = (115, 10)$



	get	see	use	hear	eat	kill
knife	51	20	84	0	3	0
cat	52	58	4	4	6	26
dog	115	83	10	42	33	17
boat	59	39	23	4	0	0
cup	98	14	6	2	1	0
pig	12	17	3	2	9	27
banana	11	2	2	0	18	0

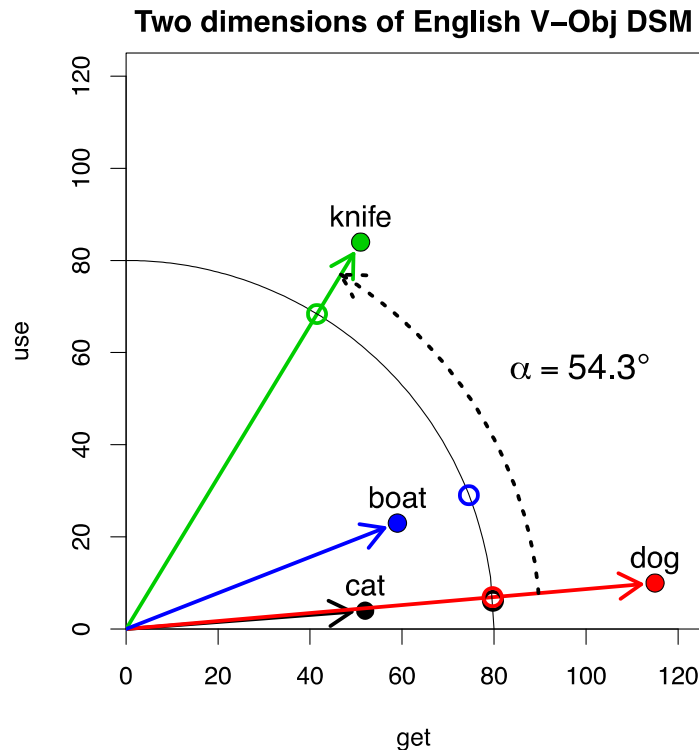
co-occurrence matrix $\mathbf{M}$

An example with interpretable vectors:

- each number represent the co-occurrence of a noun-verb pair

Distributed Representations: Geometric Interpretations #2

- ▶ vector can also be understood as arrow from origin
- ▶ direction more important than location
- ▶ use angle α as distance measure
- ▶ or normalise length $\|\mathbf{x}_{\text{dog}}\|$ of arrow



	get	see	use	hear	eat	kill
knife	51	20	84	0	3	0
cat	52	58	4	4	6	26
dog	115	83	10	42	33	17
boat	59	39	23	4	0	0
cup	98	14	6	2	1	0
pig	12	17	3	2	9	27
banana	11	2	2	0	18	0

co-occurrence matrix M

An example with interpretable vectors:

- each number represent the co-occurrence of a noun-verb pair

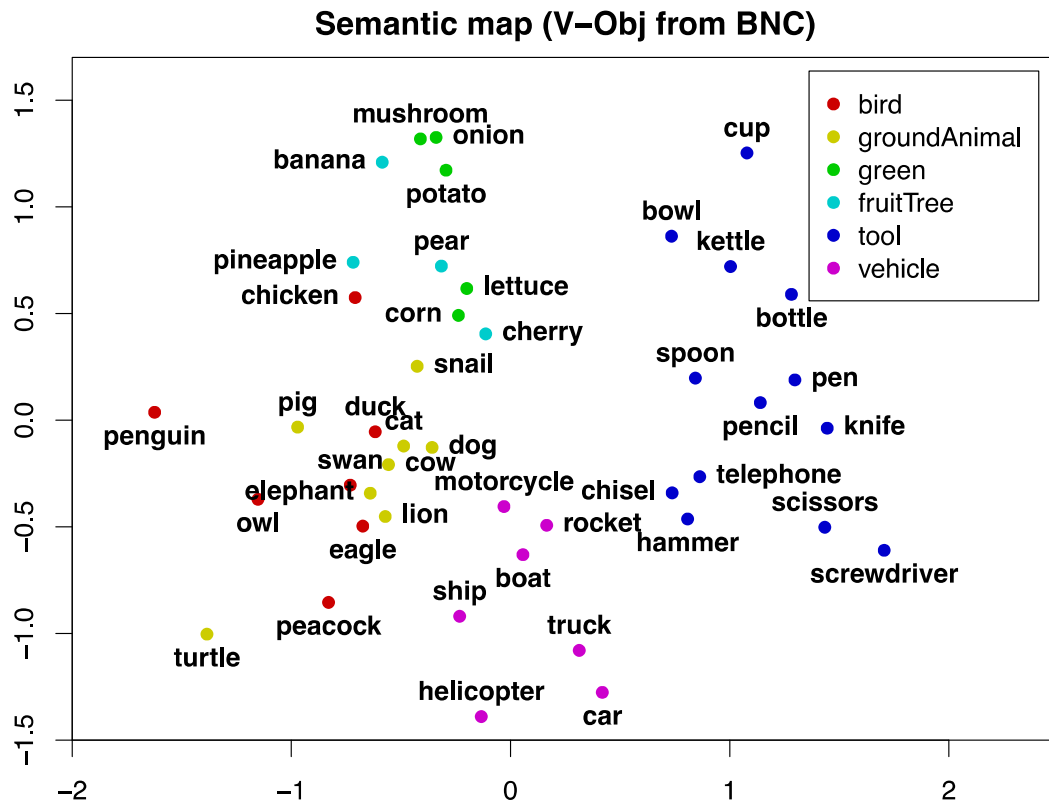
Skipping some details... it's convenient to transform word collocation counts into “better shaped”, often dense, vectors

	get	see	use	hear	eat	kill
knife	0.027	-0.024	0.206	-0.022	-0.044	-0.042
cat	0.031	0.143	-0.243	-0.015	-0.009	0.131
dog	-0.026	0.021	-0.212	0.064	0.013	0.014
boat	-0.022	0.009	-0.044	-0.040	-0.074	-0.042
cup	-0.014	-0.173	-0.249	-0.099	-0.119	-0.042
pig	-0.069	0.094	-0.158	0.000	0.094	0.265
banana	0.047	-0.139	-0.104	-0.022	0.267	-0.042

Term = word, lemma, phrase, morpheme, word pair, ...

Distributed Representations: Geometric Interpretations #2

- ▶ vector can also be understood as arrow from origin
- ▶ direction more important than location
- ▶ use angle α as distance measure
- ▶ or normalise length $\|x_{\text{dog}}\|$ of arrow



Distributed Representations: Recap

- Distributed representations: represent meaning of words, sentences, [...] with **vectors of numbers**
 - E.g., '**bardiwac**' = [0.1, 0.5, 1.5, 0.8, 3.2]
- These vectors are derived from the distribution of words in the language and have fixed dimension
- The individual numbers may not have an interpretation, but:
 - They are computed in order to **optimize the performance in some task**
 - E.g., predict '**bardiwac**' using the vectors of the words appearing nearby '**bardiwac**' in a set of (many!) sentence
 - They show interesting **regularities**
 - E.g., the vector of '**bardiwac**' will be similar to the vectors of similar words such as 'grape'. 'wine', 'beverage', etc.

How Do We Get Meaning Vectors?

■ Data:

- A large text corpus (large: Wikipedia, web crawl, ++)

■ Methods

- Co-occurrence statistics ~ word collocations

- words are represented from the contexts in which they appear (possibly with part of speech, e.g., noun-verb)

- Word embeddings

- vectors are learned while solving a prediction task

- Language models

- models learn to predict the next token in a sequence
- internal representations also encode semantic information

Distributional Semantics in a Nutshell #2

~ Cloze: guess which kind of thing [MASK1] and [MASK2] refer to

- Marco has married [MASK1].
 - Marco and [MASK1] have broke up and divorced
 - [MASK1] is now in a relationship with [MASK2]
-
- What do you believe about [MASK1] and [MASK2]?
 - What do you can **not** guess about [MASK1] and [MASK2]?
 - What do you may have additionally guessed in 1900 about [MASK1] and [MASK2]?

Distributional Semantics in a Nutshell #3

~ Next word prediction: guess what comes next [...]

- Marco and Sara have [...]
- Marco and Sara have broken [...]
- Marco and Sara have broken up [...]
- Marco and Sara have broken up and [...]

→ Marco and [...] ?

In short: what we have discussed in the previous slide can be apply to next word prediction

Distributions vs. definitions / rules

LANGUAGE MODELING

Language Models

- A language model assigns a probability to linguistic events such as:
 - a token
 - a sequence of tokens
 - the next token given previous tokens
- In the standard autoregressive setting:

$P(\text{I saw a cat on}) =$

$P(\text{I}) \cdot P(\text{saw}|\text{I}) \cdot P(\text{a}|\text{I saw}) \cdot P(\text{cat}|\text{I saw a}) \cdot P(\text{on}|\text{I saw a cat})$

└──┘ └──┘

Probability of I saw a cat

Probability of on given I saw a cat



Language Models

- A language model assigns a probability to linguistic events such as:
 - a token
 - a sequence of tokens
 - the next token given previous tokens
- In the standard autoregressive setting:

$$P(w_1, \dots, w_n) = \prod_{t=1}^n P(w_t \mid w_1, \dots, w_{t-1})$$

- Given a prefix $w_1, \dots, w_{t-1}$, the model estimates the probability distribution of the next token w_t .

WRAP-UP

Two Paradigms of Semantics

■ Symbolic / logic-based

- meaning through definitions, axioms, rulesaxioms and rules
- interpretable symbolic structures
- inference through reasoning

■ Distributional / learned

- meaning through patterns of use in data
- dense vector representations
- inference through numerical computation

Some Terminological Remarks

- The following terms may be used in different contexts with small variations but refer to very similar concepts
 - Vector-based representations
 - Because: they are numerical vectors
 - Distributed representations
 - Because: they are not discrete and distributed across different vector components
 - Dense representations
 - Because: each component has a value
 - Multi-dimensional representations
 - Because: each vector component represents a dimension
 - Embeddings
 - Because: they embed representations of discrete symbols, e.g., a word, an entity, into a vector space, i.e., a vector in this space
 - Sub-symbolic representations
 - Because: components are numbers, not symbols with a discrete symbolic interpretation